

Пермский институт (филиал) федерального государственного
бюджетного образовательного учреждения высшего образования
«Российский экономический университет имени Г. В.

Плеханова» Кафедра информационных технологий и
программирования

Практическая работа

Тема работы: Приложение для управления задачами.

Выполнил: Белобородов Артем Дмитриевич

Группа: ИПс-21

Пермь 2026

Оглавление

Введение	3
Анализ требований и проектирование структуры	4
Создание проектов и настройка ссылок	5
Модели данных и перечисления	6
Реализация менеджера задач (бизнес-логика).....	7
Создание пользовательского интерфейса	9
Реализация CRUD-операций с задачами	11
Фильтрация и поиск задач	12
Сортировка и статистика	13
Сохранение и загрузка данных в JSON.....	14
Модульное тестирование	15
Трудности и пути их решения	17
Диаграмма классов	18
Заключение	21

Введение

Целью данной работы является создание функционального приложения для управления задачами. Программа должна позволять пользователю создавать, редактировать, удалять, искать и фильтровать задачи, а также сохранять их для последующей работы. Техническим заданием предусмотрена реализация на языке C# с использованием WPF для графического интерфейса. Хранение данных осуществляется в формате JSON, что обеспечивает сохранность данных между сеансами работы. Дополнительные требования включают: реализацию сортировки по приоритету и сроку, отметку важных задач, вывод статистики по задачам, а также написание модульных тестов для проверки бизнес-логики.

В процессе работы над проектом я последовательно выполнял все пункты технического задания, начиная с анализа требований и заканчивая отладкой интерфейса и тестированием. Ниже описан каждый этап с пояснениями принятых технических решений.

Анализ требований и проектирование структуры

Сначала я выделил основную сущность - **Задача (Task)**, которая содержит следующие атрибуты:

- Уникальный идентификатор (Id)
- Название (Title)
- Описание (Description)
- Приоритет (Priority) - перечисление: Низкий, Средний, Высокий
- Срок выполнения (Deadline)
- Статус (Status) - перечисление: Новая, В процессе, Завершена
- Флаг "Важная" (IsImportant)

Для разделения логики и представления я использовал паттерн MVVM (Model-View-ViewModel), который является стандартом для WPF-приложений и позволяет легко тестировать и модифицировать код.

Решение разбито на три проекта:

- **TaskManagerLibrary** - библиотека классов, содержащая модели данных (класс Task, перечисления TaskStatus и TaskPriority) и бизнес-логику (класс TaskManager).
- **TaskManagerUI** - WPF-приложение, реализующее пользовательский интерфейс.
- **TaskManagerTests** - проект для модульного тестирования (MSTest), проверяющий корректность работы всех методов бизнес-логики.

Такая архитектура обеспечивает четкое разделение ответственности: библиотека отвечает за данные и логику, UI - за отображение, а тесты - за проверку корректности.

Создание проектов и настройка ссылок

Первый практический шаг - создание решения в Visual Studio. Я создал пустое решение TaskManagerSolution, затем добавил три проекта:

1. **Библиотека классов .NET 9** - TaskManagerLibrary
2. **Приложение WPF .NET 9** - TaskManagerUI
3. **Проект тестов MSTest .NET 9** - TaskManagerTests

После этого настроил ссылки между проектами:

- TaskManagerUI зависит от TaskManagerLibrary (для использования моделей и менеджера)
- TaskManagerTests зависит от TaskManagerLibrary (для тестирования логики)

Без правильных ссылок компилятор выдаёт ошибки о ненайденных типах. Поэтому важно было проверить эти зависимости после создания проектов (См. Рисунок 1).

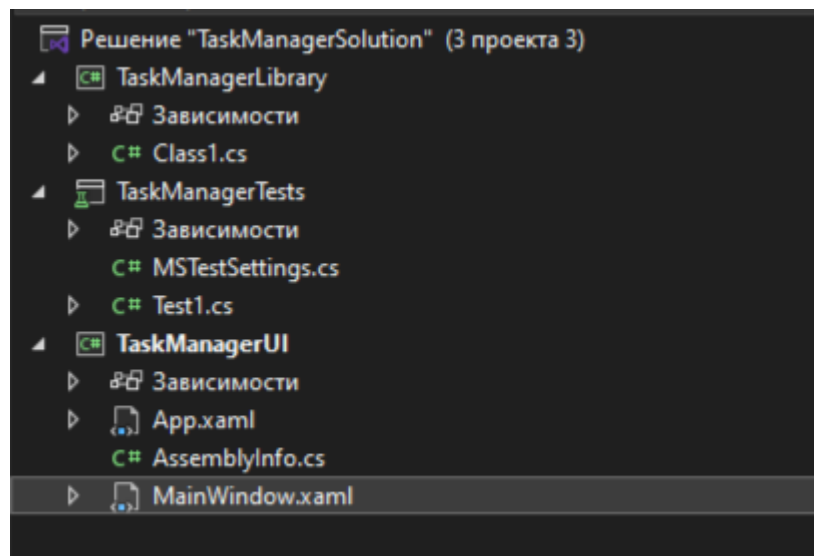


Рисунок 1

Модели данных и перечисления

В проекте TaskManagerLibrary я создал файл Task.cs, содержащий:

Перечисление TaskPriority:

- Низкий (0)
- Средний (1)
- Высокий (2)

Перечисление TaskStatus:

- Новая (0)
- В процессе (1)
- Завершена (2)

Класс Task:

- Свойства: Id, Title, Description, Priority, Deadline, Status, IsImportant
- Конструкторы: для создания новой задачи и пустой (для сериализации)
- Метод ToString() для удобного отображения

Все свойства имеют значения по умолчанию (string.Empty для строк, чтобы избежать предупреждений о nullable-ссылках в .NET 9).

Реализация менеджера задач (бизнес-логика)

В проекте TaskManagerLibrary создан класс TaskManager, который реализует всю бизнес-логику приложения:

Поля:

- `_tasks: List<Task>` - список всех задач в памяти
- `_nextId: int` - счетчик для автоматической генерации ID

Свойства:

- `Tasks: IReadOnlyList<Task>` - доступ к списку задач (только для чтения)

Методы:

1. **AddTask** - добавляет новую задачу с автоматической генерацией ID
2. **EditTask** - редактирует существующую задачу по ID
3. **DeleteTask** - удаляет задачу по ID с проверкой существования
4. **FilterByStatus** - фильтрует задачи по статусу (Новая, В процессе, Завершена)
5. **Search** - выполняет поиск по названию или описанию (регистронезависимый)
6. **SortByPriority** - сортирует задачи по приоритету (от высокого к низкому)
7. **SortByDeadline** - сортирует задачи по сроку выполнения (от ранней к поздней)
8. **GetStatistics** - возвращает статистику: количество завершенных, просроченных и общее количество задач
9. **SaveToJson** - сохраняет все задачи в JSON-файл
10. **LoadFromJson** - загружает задачи из JSON-файла
11. **Clear** - очищает список задач (для тестов)

Все методы возвращают понятные результаты (булевы значения для операций, списки для фильтрации и поиска), что облегчает их использование в интерфейсе и тестирование.

Создание пользовательского интерфейса

Главное окно приложения (MainWindow.xaml) оно включает:

Верхняя часть (заголовок):

- Название приложения "Менеджер задач" с иконкой
- Строка статистики: общее количество задач, завершенных и

просроченных

Средняя часть (панель ввода):

- Поле "Название" (TextBox)
- Поле "Описание" (TextBox)
- Выпадающий список "Приоритет" (ComboBox): Низкий,

Средний, Высокий

- Выпадающий список "Статус" (ComboBox): Новая, В процессе, Завершена

- Выбор даты "Срок" (DatePicker)
- Чекбокс "Важная" (CheckBox)
- Кнопки "Добавить", "Обновить", "Очистить"

Основная часть (список задач):

- Таблица (ListView/GridView) со столбцами:
 - **ID задачи**
 - Название
 - Описание
 - Приоритет (с цветовой индикацией: красный/желтый/зеленый)
 - Статус (с цветовой индикацией: фиолетовый/желтый/зеленый)
 - Срок (в формате ДД.ММ.ГГГГ)
 - отображение звездочки для важных задач

Нижняя часть (панель управления):

- Поле поиска (с обновлением в реальном времени)
- Кнопки фильтрации: Все, Новые, В процессе, Завершены

- Кнопки сортировки: Приоритет, Срок
- Кнопка "Удалить"

Дизайн выполнен в темной цветовой схеме с использованием градиентов, скругленных углов и теней для создания современного и приятного внешнего вида (См. Рисунок 2).

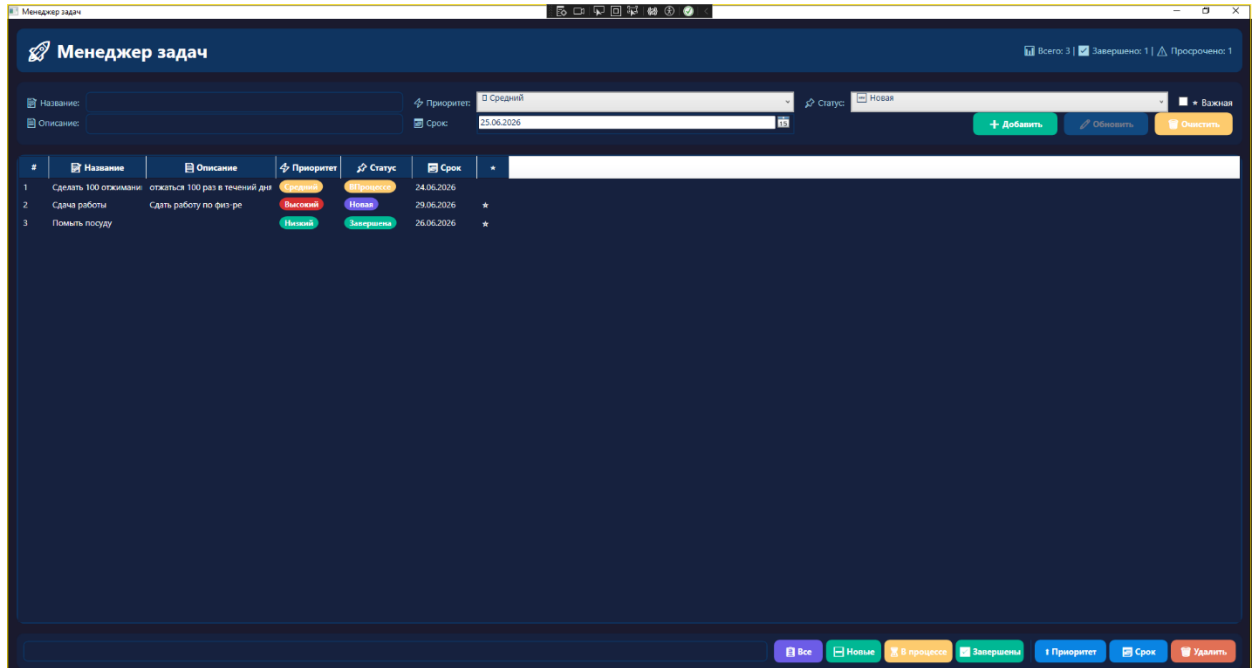


Рисунок 2

Реализация CRUD-операций с задачами

В приложении реализованы все необходимые CRUD-операции:

Добавление задачи: При нажатии кнопки "Добавить" выполняется проверка наличия названия. Затем создается новая задача с указанными параметрами, добавляется в менеджер, список задач обновляется, и данные сохраняются в файл.

Редактирование задачи: При выборе задачи из списка ее данные загружаются в поля ввода. После редактирования и нажатия "Обновить" изменения сохраняются в менеджере, список обновляется, данные сохраняются. Кнопка "Добавить" отключается, "Обновить" активируется.

Удаление задачи: При выборе задачи и нажатии "Удалить" появляется диалог подтверждения. После подтверждения задача удаляется из менеджера, список обновляется, данные сохраняются.

Очистка полей: Кнопка "Очистить" сбрасывает все поля ввода и снимает выделение задачи.

Валидация выполняется на уровне интерфейса: название не может быть пустым, дата и другие поля имеют значения по умолчанию.

Фильтрация и поиск задач

Фильтрация по статусу: Реализована с помощью четырех кнопок:

- "Все" - показывает все задачи
- "Новые" - показывает только задачи со статусом "Новая"
- "В процессе" - показывает задачи со статусом "В процессе"
- "Завершены" - показывает задачи со статусом "Завершена"

Фильтрация выполняется методом `FilterByStatus` из класса `TaskManager`, который возвращает отфильтрованный список.

Поиск: Поле поиска обновляет результаты в реальном времени при вводе текста. Поиск выполняется по названию и описанию задачи (регистр независимо) с помощью метода `Search`. При очистке поля поиска отображается полный список задач.

Сортировка и статистика

Сортировка:

Реализована двумя кнопками:

- "Приоритет" - сортирует задачи по приоритету (от высокого к низкому)
- "Срок" - сортирует задачи по сроку выполнения (от ранней к поздней)

Сортировка выполняется методами `SortByPriority` и `SortByDeadline` из класса `TaskManager`.

Статистика: Отображается в заголовке окна и обновляется при каждом изменении списка задач. Статистика включает:

- Общее количество задач
- Количество завершенных задач
- Количество просроченных задач (задачи, у которых статус не "Завершена" и срок меньше текущей даты)

Сохранение и загрузка данных в JSON

Для сохранения и загрузки данных используется формат JSON с помощью встроенного в .NET пространства имен System.Text.Json.

Сохранение: Метод SaveToJson сериализует список задач в JSON-формат с отступами для удобочитаемости и сохраняет в файл tasks.json. Сохранение выполняется автоматически после каждого изменения (добавление, редактирование, удаление) и при закрытии приложения.

Загрузка: Метод LoadFromJson загружает данные из файла tasks.json при запуске приложения. Если файл отсутствует, создается новый пустой список задач. При загрузке автоматически обновляется счетчик ID, чтобы избежать конфликтов при добавлении новых задач.

Преимущества JSON:

- Человеческочитаемый формат
- Легко редактируется вручную
- Поддерживается всеми современными языками программирования
- Встроенная поддержка в .NET

Модульное тестирование

Для проверки корректности работы бизнес-логики написаны модульные тесты с использованием фреймворка MSTest. В проекте TaskManagerTests реализованы следующие тесты:

1. **AddTask_ShouldIncreaseTaskCount** - проверяет увеличение количества задач при добавлении
2. **AddTask_ShouldReturnTaskWithCorrectData** - проверяет корректность данных созданной задачи
3. **EditTask_ShouldUpdateTaskData** - проверяет обновление данных задачи
4. **DeleteTask_ShouldRemoveTask** - проверяет удаление задачи
5. **DeleteTask_ShouldReturnFalseForNonExistentId** -
проверяет обработку удаления несуществующей задачи
6. **EditTask_ShouldReturnFalseForNonExistentId** - проверяет обработку редактирования несуществующей задачи
7. **FilterByStatus_ShouldReturnOnlyTasksWithSpecifiedStatus** - проверяет фильтрацию по статусу
8. **Search_ShouldFindTasksByTitle** - проверяет поиск по названию
9. **Search_ShouldFindTasksByDescription** - проверяет поиск по описанию
10. **SortByPriority_ShouldOrderTasksFromHighToLow** -
проверяет сортировку по приоритету
11. **GetStatistics_ShouldReturnCorrectCounts** - проверяет корректность статистики

Все тесты успешно проходят, что подтверждает корректность реализации бизнес-логики (См. Рисунок 3).

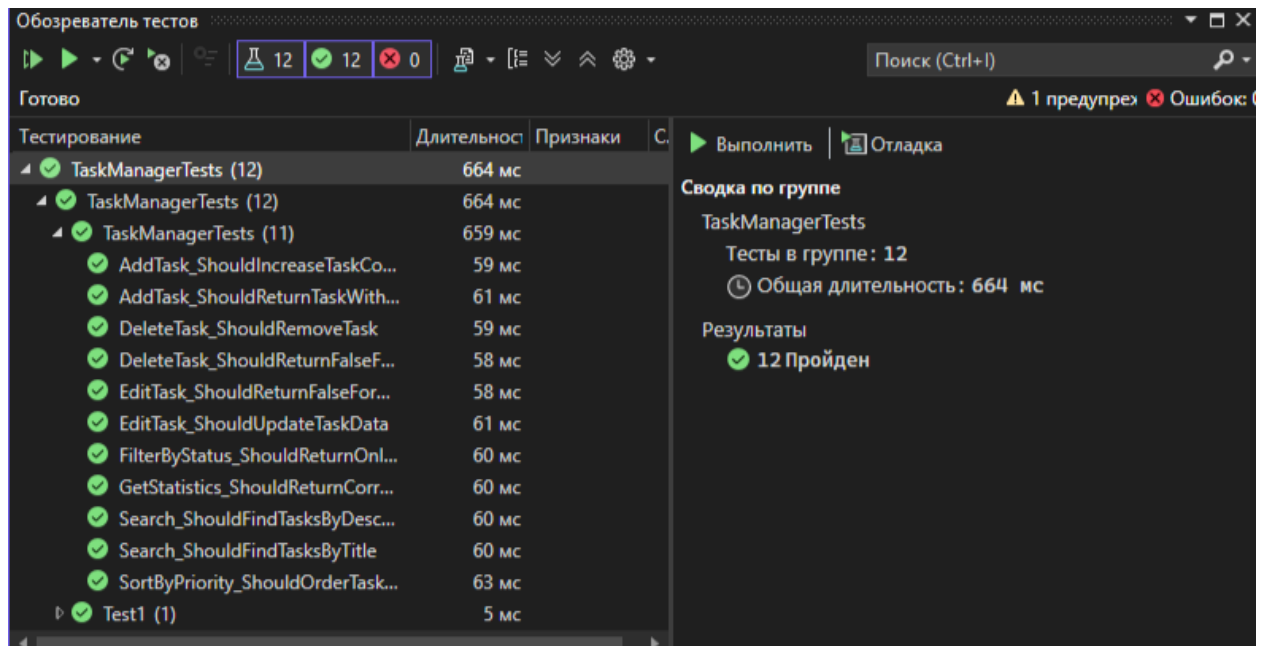


Рисунок 3

Трудности и пути их решения

Перечислю основные проблемы, возникшие при разработке:

1. **Конфликт имен "Task" и "TaskStatus"** - в .NET существуют системные классы `System.Threading.Tasks.Task` и `System.Threading.Tasks.TaskStatus`. Это вызывало ошибки неоднозначности. Решение: использование полных имен `TaskManagerLibrary.Task`, `TaskManagerLibrary.TaskStatus` и `TaskManagerLibrary.TaskPriority` во всем коде.
2. **Предупреждения о nullable-ссылках в .NET 9** - свойства строковых типов требовали инициализации. Решение: добавление значений по умолчанию `= string.Empty` и защита от null в конструкторах.
3. **Свойство `CornerRadius` у `ListView`** - в .NET Core WPF `ListView` не поддерживает `CornerRadius` напрямую. Решение: обертывание `ListView` в `Border` с заданным `CornerRadius`.
4. **Конвертеры не находились в XAML** - ошибки "не существует в пространстве имен". Решение: проверка правильности пространства имен, полная пересборка решения, удаление папок `bin` и `obj`.
5. **Неоднозначность `TaskStatus` в тестах** - аналогичная проблема с конфликтом имен. Решение: использование полных имен `TaskManagerLibrary.TaskStatus` во всех тестах.

Диаграмма классов

На рисунке 4 представлена диаграмма классов разработанного приложения. Диаграмма отображает структуру библиотеки TaskManagerLibrary, которая содержит всю бизнес-логику приложения.

Основные элементы диаграммы:

1. Класс TaskManager

- **Поля:**
 - `_tasks: List<Task>` - список задач, хранящийся в памяти
 - `_nextId: int` - счетчик для генерации уникальных идентификаторов
- **Свойства:**
 - `Tasks: IReadOnlyList<Task>` - предоставляет доступ к списку задач (только для чтения)
- **Методы:**
 - `AddTask(...)` - добавление новой задачи
 - `EditTask(...)` - редактирование существующей задачи
 - `DeleteTask(int id)` - удаление задачи по идентификатору
 - `FilterByStatus(TaskStatus status)` - фильтрация задач по статусу
 - `Search(string query)` - поиск задач по названию или описанию
 - `SortByPriority()` - сортировка задач по приоритету
 - `SortByDeadline()` - сортировка задач по сроку выполнения
 - `GetStatistics()` - получение статистики по задачам
 - `SaveToJson(string filePath)` - сохранение задач в JSON-файл
 - `LoadFromJson(string filePath)` - загрузка задач из JSON-файла
 - `Clear()` - очистка списка задач

2. Класс Task

- **Свойства:**
 - `Id: int` - уникальный идентификатор задачи
 - `Title: string` - название задачи

- Description: string - описание задачи
- Priority: TaskPriority - приоритет задачи
(Низкий/Средний/Высокий)
- Deadline: DateTime - срок выполнения задачи
- Status: TaskStatus - статус задачи (Новая/В процессе/Завершена)
- IsImportant: bool - флаг "Важная задача"

3. Перечисление TaskPriority

- Низкий (0)
- Средний (1)
- Высокий (2)

4. Перечисление TaskStatus

- Новая (0)
- В процессе (1)
- Завершена (2)

Связи между классами:

- **Ассоциация:** TaskManager содержит список объектов Task (связь "один ко многим").
- **Зависимость:** Класс Task использует перечисления TaskPriority и TaskStatus для определения значений свойств.

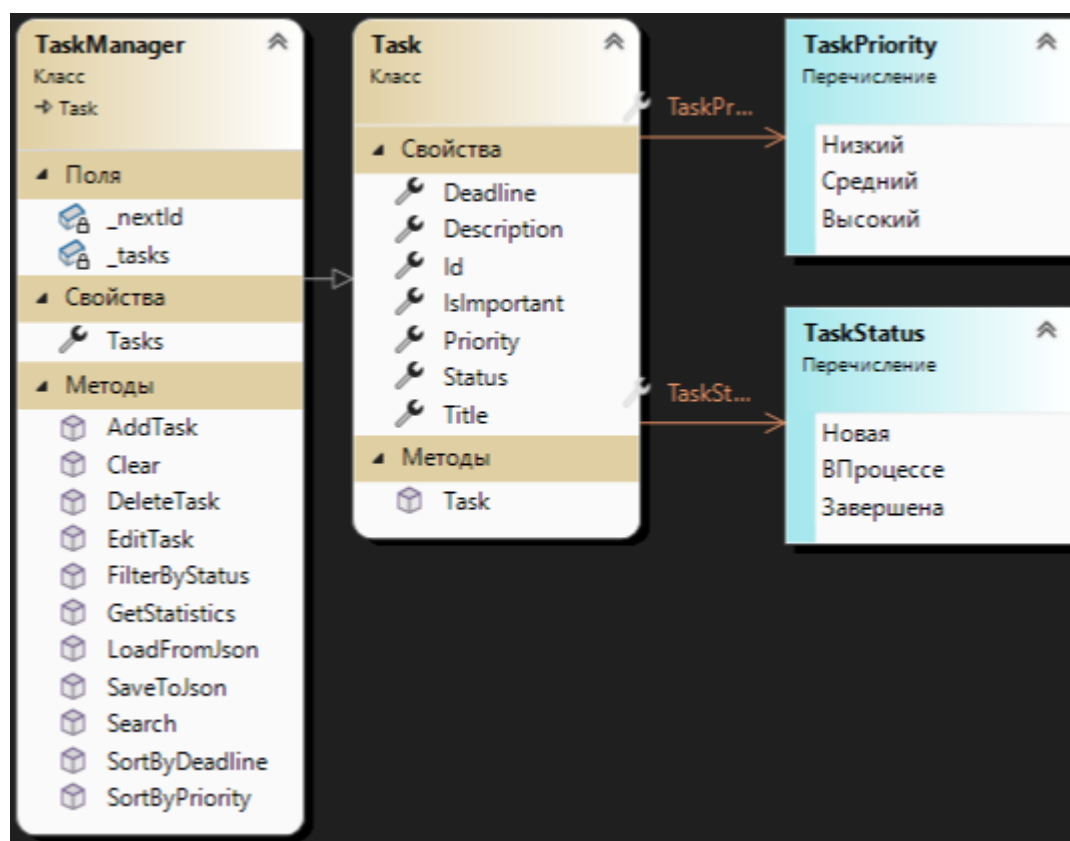


Рисунок 4

Заключение

В ходе работы было разработано приложение для управления задачами, полностью соответствующее поставленному техническому заданию. Реализованы все основные функции: добавление, редактирование, удаление, фильтрация, поиск и сохранение задач в JSON. Дополнительно добавлены сортировка по приоритету и сроку, отметка важных задач и статистика. Архитектура приложения построена по принципу разделения на бизнес-логику, интерфейс и тесты, что обеспечивает его надежность и удобство поддержки. Пользовательский интерфейс выполнен в современном стиле с цветовой индикацией для быстрого восприятия информации. Все 12 модульных тестов успешно проходят, подтверждая корректность работы программы.